

実録音音源の原音再現を目的とした ニューラルネットワーク型音響フィルタの研究

齋藤 健吾[†]

[†] 有明工業高等専門学校 創造工学科 情報システムコース
Gauthier Lovic 研究室

Development of a Deep Learning-Based Filter for Original Sound Restoration from Real-World Audio Recordings

Kengo Saito[†]

[†] Information System Course, Department of Creative Engineering,
National Institute of Technology (KOSEN), Ariake College
Gauthier Lovic Laboratory

2026 年 1 月 27 日

Abstract – 本研究では、スマートフォン等で音源から離れた位置で録音された実録音音源に含まれる伝搬・收音特性（距離減衰、周波数特性の歪み、環境ノイズ、残響など）を補正し、音源近傍で得られる、原音に近い波形を再現するニューラルネットワーク型音響フィルタを提案する。まず、実環境データにおける入力と教師の開始時刻ずれを解消するため、相互相関に基づく遅延推定により 2 系列を自動整列し、共通区間のみを切り出す前処理を行った。次に、短時間フーリエ変換 (Short-Time Fourier Transform; STFT) により得た複素スペクトルを入力として、2 次元 U-Net で周波数領域の補正スペクトルを推定し、逆 STFT (iSTFT) で時間波形へ復元する。学習では複素スペクトル差分に加え、Log-Mag 損失、振幅損失、SNR 損失の重み付き和を用い、これに加え、教師がほぼ無音の時間 – 周波数領域では残留成分を強く罰する無音強制項を導入した。44.1 [kHz] の実録音データ 50 ペア（訓練 40 ペア/検証 10 ペア）を用いて学習し、検証データから抽出した 10 ペアで評価した結果、無音強制項付き損失が SNR 改善量 ΔE_{snr} および無音領域の残留成分パワー低減量 ΔE_{sil} の両方で最良となり、SNR は平均で約 5.2 [dB] 改善した。また、教師振幅の下位 10% 領域における残留成分パワーは最大で約 20 [dB] 低減し、無音区間のノイズフロア抑制に有効であることを確認した。更なる改善のためには、検証 10 ペア全体での評価と分散の集計、より多様な收音環境・音源への一般化、および知覚品質を考慮した評価指標・損失関数の導入が検討される。

Keywords: 実録音音源, 音響フィルタ, 相互相関, STFT, 周波数領域補正, ニューラルネットワーク, U-Net

目次

1	序論	1
1.1	背景と目的	1
1.2	課題設定	1
1.3	簡易的な手法・従来研究との差別化・研究意義	1
2	関連研究	1
3	信号モデルと課題設定	2
3.1	信号モデルの設定	2
3.2	フロー全体のモデル化	2
3.2.1	時間領域の畳み込み作用素 T_h	2
3.2.2	ノイズ付加作用素 $\oplus v$	2
3.2.3	STFT 作用素 S	3
3.2.3.1	導入意義	3
	・ 他手法との比較	3
	・ STFT の利点	3
3.2.3.2	短時間フーリエ変換 S (STFT)	3
	・ STFT の基本的な考え方	3
	・ 定義	3
	・ STFT の出力	4
	・ 分解能	4
	・ セグメント化	4
3.2.4	周波数領域の写像 f	4
3.2.5	逆短時間フーリエ変換 $S^\dagger$ (iSTFT)	4
4	データセットと前処理	5
4.1	収録条件	5
4.2	相互相関による時間揃え	5
4.2.1	相互相関を行う目的	5
4.2.2	相互相関 $R_{xy}(\tau)$ の定義	5
4.2.3	共通区間の切り出し	5
4.2.4	プログラム上での実装	5
5	モデル構成	6
5.1	モデルの概要と入出力形式	6
5.2	損失関数と最適化	6
5.2.1	損失関数の定義	6
5.2.2	各損失の役割	7
5.2.2.1	複素スペクトル損失	7
5.2.2.2	Log-Mag 損失	7
5.2.2.3	SNR 損失	7
5.2.2.4	無音強制項	7
5.2.3	最適化	8

5.2.4	前処理－学習アルゴリズムのフロー	8
5.3	期待リスク最小化	8
5.4	ネットワーク詳細	9
6	実験設定	10
6.1	動作環境	10
6.2	データセットと学習設定	10
6.2.1	複素スペクトル損失低減量 ΔE_{cplx}	10
6.2.2	SNR 改善量 ΔE_{snr}	10
6.2.3	無音領域の残留成分低減量 ΔE_{sil}	10
7	結果	10
7.1	複素スペクトル損失のみ	10
7.2	無音強制項なし重み付き損失	10
7.3	無音強制項付き損失	12
7.3.1	損失関数の比較	12
8	考察	12
8.1	全体性能と SNR 改善	12
8.2	無音評価指標の導入と効果	12
8.3	教師音声の残留ノイズによる影響	13
8.4	無音強制項の限界と今後の検証	13
9	結論と今後の課題	13
9.1	結論	13
9.2	今後の課題	13
	謝辞	13
	参考文献	13

1 序論

1.1 背景と目的

講義録画やリモート会議では、音源から離れた地点に置かれたマイクロホンで収録されることが多い。このとき距離減衰、室内残響、環境ノイズが重畳し、特定帯域の減衰や時間的に変化する歪み^{*1}などが生じる。雑音は一般に環境要因や機械的要因など、様々な要因に起因する。ホワイトノイズ^{*2}やピンクノイズ^{*3}、空調やファンの音などの周期的な機械音や人為的な話し声など、多様なノイズが混在する。このような劣化は可聴性や明瞭度を低下させるだけでなく、自動字幕生成や音声認識などの後段処理の性能にも影響する。教育・医療・公共アーカイブなど、音源の質が情報理解に直結する場面も増えており、高度な音声改善技術の重要性は高まっている。

本研究では、こうしたノイズを含む実録音音源から、明瞭でノイズの少ない原音を再現する音響フィルタの構築を目指す。ここで「実録音音源」とは、実際の空間と機器で録られた音（ノイズや残響を含む）を指し、「原音」とは音源近傍で理想的に録られた参照音を指す。

1.2 課題設定

本研究では、離れた地点で録音したノイズや伝搬・收音特性を含む音源と音源近傍で録音した教師音源のペアを用いて、教師あり学習により、ニューラルネットワーク型の音響フィルタを構築する。

これらの訓練データの組は手動での録音によって得られるが、録音開始時刻のずれが存在すると、異なる時間軸での対応関係が学習されてしまい、学習が安定しないことが予期される。しかし、このずれを解消するにあたり、簡易的な整列アルゴリズム^{*4}や手動での整列では品質・作業効率ともに不十分である。そこで本研究では、相互相関を用いた自動時間揃え処理を導入し、学習用データの前処理を行う。

次に、時間－周波数表現を用いて劣化特性を補正するニューラルネットワークモデルを構築・学習し、実環境ノイズ下での原音再現を目指す。

1.3 簡易的な手法・従来研究との差別化・研究意義

本研究を行う目的として、簡易的な手法では解決できない課題に対応できる点が挙げられる。例として、録音した音源が低音域で大きく減衰している場合を考える。この際、考えられる簡易的な音響フィルタとして、ローパスフィルタやイコライザなどの

解析的に周波数や振幅を変更する手法を挙げることができる。しかしながら、ローパスフィルタによる単純な帯域強調では、ノイズ成分も同時に増幅されてしまい、音質改善にはつながらない。イコライザによる帯域補正なども同様の理由で、ノイズ特性が時間的に変動する場合には十分な効果を得られない。

本研究では、ニューラルネットワークを用いて、実録音音源の複雑な劣化特性を学習し、原音再現を目指すことで、これらの従来手法では対応が難しい課題に対処することを試みた。

2 関連研究

実録音音源の補正に対する従来手法として、Wiener フィルタ [1] やノイズ抑圧フィルタ (spectral subtraction) [2] が用いられてきた。Wiener フィルタは、信号とノイズの統計的特性を用いて最適な線型フィルタを設計する手法であり、ノイズ抑圧フィルタは、スペクトルサブトラクション法に基づき、観測信号から雑音成分を推定し、時間－周波数領域で減算することで音質を改善する音声強調の手法である。これは、Fig.1 の vague 部で示した $T_h(\mathbf{y})$ と $\mathbf{v}$ の分離に対応し、 $\mathbf{v}$ の推定に基づき $T_h(\mathbf{y})$ を回復するという考え方に近い。これらの手法は計算コストが低くリアルタイム処理に適しているが、ノイズ特性が時間的に変動する場合や、各環境によりインパルス応答が大きく異なる場合には、十分な効果を得られないことがある。

雑音抑圧や帯域補完の分野では、STFT 空間でのマスク推定 [4] や畳み込みニューラルネットワーク (Convolutional Neural Network; CNN) を用いたスペクトル補正 [6] が提案されている。また、敵対的学習^{*5}や拡散モデル^{*6}による高音質化も報告されている [12, 13]。他方、時間領域直接処理では位相情報の扱いが難しく、周波数領域処理では時間分解能とのトレードオフが生じる。すなわち、波形を直接扱う場合は位相に相当する時間的整合が崩れると歪みとして現れやすい。また、STFT などの時間－周波数表現では窓長を長くすると周波数分解能は上がるが時間分解能は下がり、逆も同様であるため、両者を同時に高めにくい。Conv-TasNet のように時間領域でマスクを学習する手法 [8] や、U-Net による歌声分離 [9]、Wave-U-Net [10]、Demucs [11] のようにエンドツーエンドで畳み込み復元を行う手法、SEGAN [12] のように生成モデルを用いる手法も存在する。また、複素スペクトルを直接扱う DCUNet

^{*1} 距離減衰や残響、ノイズの影響により、周波数帯域ごとに異なる減衰や遅延が生じる現象。

^{*2} 全周波数にわたり均一なパワー（≒ 振幅）を持つノイズ。

^{*3} 周波数が高くなるほどパワーが減少するノイズ。

^{*4} 例として、ピーク地点での合わせや、2 乗誤差の最小化による全体的な合わせなどが挙げられる。

^{*5} 生成器と識別器を競合的に学習させることで、より高品質なデータ生成を行う生成モデルの一種。すなわち、生成器が補正スペクトルを生成し、識別器が原音スペクトルと補正スペクトルを識別する役割を担う。

^{*6} ノイズを段階的に除去することでデータを生成する生成モデルの一種。画像生成分野で高精度な生成が可能であることが報告されており、音声生成分野にも応用されている。

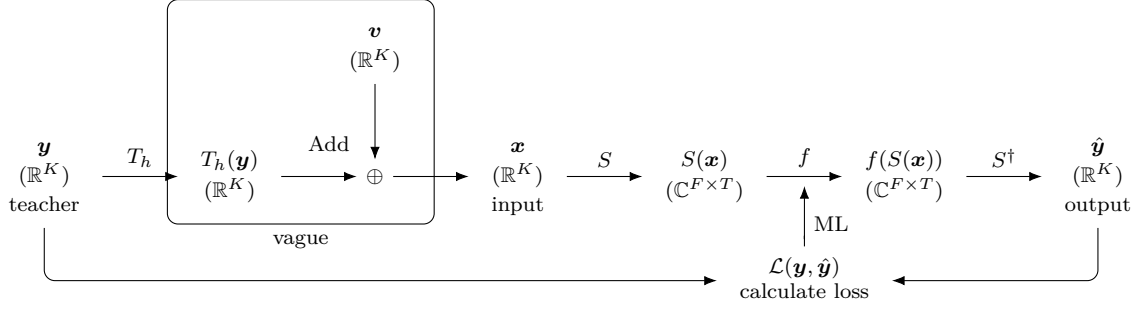


Fig. 1 信号の解釈と学習全体のフロー. K は波形長, F は周波数ビン数, T はフレーム数を表す. 括弧内はその信号が属する空間を示しており, 矢印に併記している記号は関数・写像を表す. また, 図中 ML は Machine Learning を表し, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ への矢印は損失関数への引数, ML への矢印は, 写像 f を $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ によって更新することを示している. vague で囲われた部分は, ノイズ混入プロセスの解釈である.

系 [19] や DCCRN[20], PHASEN[21] は位相補正も同時に行う.

本研究は, 相互相関による自動アライメント^{*7}を行うことにより, 周波数領域補正に集中できる 2 次元 U-Net を作成し, 一つの音声を幾つかに区切り, 複数の音声に分割することにより, 限られた実録音データでも安定に学習できるよう構成した.

3 信号モデルと課題設定

3.1 信号モデルの設定

音源近傍で得られる長さ t [s] の理想波形を $\mathbf{y} := (y_0, \dots, y_{K-1}) \in \mathbb{R}^K$, 距離や環境によるノイズが乗った実録音音源の波形を $\mathbf{x} := (x_0, \dots, x_{K-1}) \in \mathbb{R}^K$ とする^{*8}. なお, ここで $K := 44,100 \times t$ であり, サンプリング周波数 44.1 [kHz] で t [s] 分のサンプル数を表す. 音響伝達を有限長インパルス応答^{*9} $\mathbf{h} := (h_0, \dots, h_{K-1}) \in \mathbb{R}^K$, 環境ノイズを $\mathbf{v} := (v_0, \dots, v_{K-1}) \in \mathbb{R}^K$ で表すとする. このとき, 音源 $\mathbf{y}$ から離れた場所で録音する波形 $\mathbf{x}$ は

$$x[n] = \sum_{k=0}^n y[n-k] h[k] + v[n], \quad n = 0, \dots, K-1 \quad (1)$$

と見做せる^{*10} (3.2.1, 3.2.2 節参照). 目標は, $\mathbf{x}$ から原音を近似する $\hat{\mathbf{y}} = S^\dagger(f(S(\mathbf{x})))$ を得て, $\mathbf{y}, \hat{\mathbf{y}}$ の差を最小化するような関数 f (Fig.1 に示す f に相当) を機械学習により構成することである.

実際には $\mathbf{y}$ と $\hat{\mathbf{y}}$ を後に述べる変換で時間-周波数表現に変換し, 複素スペクトル損失や SNR, Log-Mag 損失などを $\mathbb{R}^K$ 上ではなく $\mathbb{C}^{F \times T}$ 上で評価した. 損失関数と最適化については 5.2 節で詳述する.

^{*7} 時間軸のずれを補正する処理. 揃え, 整列とも表記される.

^{*8} 本研究では音声を 32 [bit] 浮動小数点で読み込み, 正規化して扱うため, 以降の信号は実数値ベクトル $\mathbb{R}^K$ として記述する.

^{*9} (有限の) 時間軸を持つ音響伝達関数としての表現であり, 音源からマイクロホンまでの伝搬特性を表す.

^{*10} ベクトルの添字を配列の添字のように表している.

3.2 フロー全体のモデル化

本節では, 信号の型や作用素を数式的に表現・定義し, 全体を俯瞰できるよう, 時間領域の畳み込みや周波数領域の補正処理を作用素の合成としてまとめておく.

3.2.1 時間領域の畳み込み作用素 T_h

信号 $\mathbf{y} \in \mathbb{R}^K$ とインパルス応答 $\mathbf{h} \in \mathbb{R}^K$ に対する離散畳み込み (Discrete Convolution) を表す作用素 $T_h: \mathbb{R}^K \rightarrow \mathbb{R}^K$ は

$$T_h(\mathbf{y}) := \mathbf{h} * \mathbf{y} \quad (2)$$

で定義され, 1 サンプル分の畳み込み $(\mathbf{h} * \mathbf{y})[n]$ は

$$(\mathbf{h} * \mathbf{y})[n] = \sum_{k=0}^n y[n-k] h[k], \quad n = 0, \dots, K-1 \quad (3)$$

で定義される. すなわち, n 番目の畳み込み $(\mathbf{h} * \mathbf{y})[n]$ は, 信号 $\mathbf{y}$ の n 番目の 1 サンプル $y[n]$ の信号のみならず, 以前 $0 \sim n-1$ の n サンプル分の信号 $y[n-k]$ ($k = 1, \dots, n$) にその時刻に対応するインパルス応答 $h[k]$ を重みとして乗じたものを全て足し合わせたものである. 情報として過去 n サンプル分の信号 $\mathbf{y}$ とインパルス応答 $\mathbf{h}$ が含まれている.

これは, T_h が時間不変線型系 (Time-Invariant Linear System; LTI) ^{*11} の作用素であることを意味し, 減衰や残響を表現できることを示している.

3.2.2 ノイズ付加作用素 $\oplus \mathbf{v}$

ノイズ付加作用素 $\oplus: \mathbb{R}^K \times \mathbb{R}^K \rightarrow \mathbb{R}^K$ は

$$\mathbf{x} = T_h(\mathbf{y}) \oplus \mathbf{v} \quad (4)$$

で定義され, 各サンプルごとに

^{*11} 入力信号に対して線型かつ時間的に変化しない応答を示すシステムのことを指す.

$$x[n] = (h * y)[n] + v[n] \quad (5)$$

$$= \sum_{k=0}^n y[n-k] h[k] + v[n], \quad n = 0, \dots, K-1 \quad (6)$$

を行う作用素であり、ノイズ v の付加を表現する。

以上により、録音された音声信号 x は時系列的に過去に依存する離散畳み込み $T_h(y)$ で減衰や残響を表現し、 v の加算でノイズを表現することができる。と解釈できる。

3.2.3 STFT 作用素 S

Fig.1 に示すように、後に述べる機械学習によって構成されたモデルを表す作用素 f に入力の音声信号 x を与える前に、時間－周波数表現へ変換する作用素 $S: \mathbb{R}^K \rightarrow \mathbb{C}^{F \times T}$ を導入する。

3.2.3.1 導入意義

■ 他手法との比較

音声信号などの離散時間信号 $x \in \mathbb{R}^K$ を周波数解析する最も基本的な手法は、有限長信号に対する離散フーリエ変換 (Discrete Fourier Transform; DFT) である。DFT は信号全体に対して周波数成分を求めるため、得られるスペクトルは「どの周波数成分が含まれるか」を示す一方で、その周波数成分が「時間的にいつ現れたか」という情報を直接は含まない。従って、時間とともにスペクトル特性が変化する非定常信号^{*12}を扱うには、周波数情報に加えて時間方向の情報も同時に保持できる表現が望ましい。

■ STFT の利点

以上の問題は、短時間フーリエ変換 (Short-Time Fourier Transform; STFT) を用いることで解決できる。STFT は、時間－周波数表現を得ることができるため、信号の時間的な変化を捉えやすい。

また、一般に機械学習においては、画像認識などで用いられる畳み込みニューラルネットワーク (Convolutional Neural Network; CNN) は精度が高くかつ計算効率の良いモデルとされる。そのため、時間 (Amplitude) である 1 次元の入力信号を時間－周波数 (Amplitude) の 2 次元の画像^{*13}として扱うことで、画像 $\mapsto$ 画像への変換を行う CNN を適用することができる。

このように、STFT を用いる利点は、帯域ごとの減衰や歪みを周波数軸上で明示的に扱える点、出力を時間－周波数平面上の行列として表現でき、畳み込み演算に適したテンソル形状に変換できる点、学

^{*12} 音声のように瞬時的にスペクトルが変動する信号。

^{*13} 実際には横－縦の順で時間－周波数軸を表現し、Amplitude は画像の輝度に相当する。

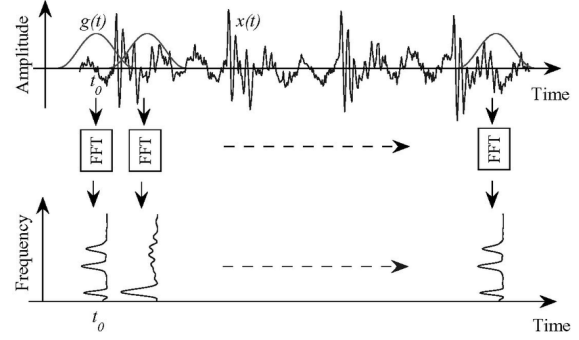


Fig. 2 STFT の例. 横軸が時間、縦軸が周波数を表し、変換後の振幅が強さを表すスペクトログラムである [3].

習画像により振幅と位相を同時に補正できる点、音声信号の問題を CNN を用いて画像処理の問題として扱える点などが挙げられる。

3.2.3.2 短時間フーリエ変換 S (STFT)

■ STFT の基本的な考え方

STFT の基本的な考え方は単純であり、信号全体に対して一度だけ DFT を適用する代わりに、信号を時間方向に有限長の区間へ分割し、各区間 (フレーム) ごとに DFT を適用する。ここでいう「時間方向に有限長の区間へ分割する」とは、窓関数を信号上で時間的に平行移動させながら掛け合わせることであり、各時刻近傍の部分信号のみを取り出す操作を指す。この操作により、各フレームのスペクトルは信号の局所区間 (窓が重なる区間) にのみ依存し、時間インデックス^{*14}付きの周波数表現が得られる。その例を Fig.2 に示す。

■ 定義

入力の離散時間実信号を $x \in \mathbb{R}^K$ とする。ここで K は信号長 (サンプル数) である。時間領域のフレームに重み付けを行う窓関数を $w = w[m]$ ($m = 0, \dots, L-1$)、隣接するフレーム間の時間シフト量を表すホップ長を H 、DFT 長 (FFT サイズ) を N_{fft} とし、本研究では入力波形が実数値であるため、片側スペクトル (非負周波数側)^{*15}を用いて周波数ビン数を $F = N_{\text{fft}}/2 + 1$ とする。窓長 L とホップ長 H から、STFT 窓の重なり具合を表すオーバーラップ率は $(L - H)/L$ で与えられる。本研究では $N_{\text{fft}} = L = 1024$, $H = 256$ より 75 [%] であり、 $F = 513$ である。これらを用いると、信号 x の STFT $S(x)$ は、時間 t , 周波数 f に対し

^{*14} STFT における時間方向の離散化されたフレーム番号を指す。

^{*15} 実数信号 $x[n] \in \mathbb{R}$ のフーリエ変換は共役対称性 $X(-\omega) = X(\omega)^*$ を満たすため、負の周波数側の情報は冗長である。そこで $f = 0, \dots, N_{\text{fft}}/2$ の非負周波数側のみを保持し、周波数ビン数は $N_{\text{fft}}/2 + 1$ としている。

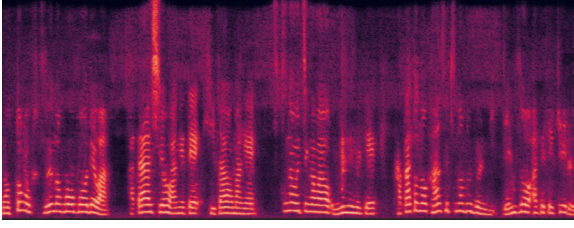


Fig. 3 STFT を行った後の時間 – 周波数表現画像の例. 横軸は時間フレーム t , 縦軸は周波数ビン f を表す. 画像は $\mathbb{C}^{F \times T}$ の元であり, 各画素は時間 t , 周波数 f に対応する複素数 $S(\mathbf{x})[f, t] \in \mathbb{C}$ の大きさ (振幅) を取り, 色の強さで表現している.

$$S(\mathbf{x})[f, t] = \sum_{m=0}^{L-1} x[tH + m] w[m] e^{-j2\pi fm/N_{\text{fft}}} \quad f = 0, \dots, F-1, t = 0, \dots, T-1 \quad (7)$$

で定義される^{*16}. ここで T は時間方向のフレーム数, つまり, 窓長 L の区間をホップ長 H で走査したときに得られる区間数であり, サンプル $x[tH + m]$ が信号範囲内に収まる最大の t に基づいて,

$$T = \left\lfloor \frac{K-L}{H} \right\rfloor + 1 \quad (8)$$

と定まる. 従って, T は時間方向の離散化数に対応する.

■ STFT の出力

STFT の出力は $S(\mathbf{x}) \in \mathbb{C}^{F \times T}$ であり, 列方向が時間フレーム $t (= 0, \dots, T-1)$, 行方向が周波数ビン $f (= 0, \dots, F-1)$ に対応する. このとき, 各成分 $S(\mathbf{x})[f, t] \in \mathbb{C}$ を STFT 係数と呼び, 時刻フレーム t と周波数ビン f における複素振幅 (振幅と位相) を表す. 実際には Fig.3 のような画像で表される.

■ 分解能

サンプリング周波数を f_s とすると, 周波数分解能は $\Delta f = f_s/N_{\text{fft}}$ で与えられる. 本研究では $f_s = 44.1$ [kHz], $N_{\text{fft}} = 1024$ より $\Delta f \approx 43.1$ [Hz] である. また, ホップ長 $H = 256$ よりフレーム間隔 (時間刻み) は $H/f_s \approx 5.80$ [ms], 窓長 $L = 1024$ よりフレーム長は $L/f_s \approx 23.2$ [ms] となる.

計算量と分解能の両立のため, 本研究では STFT 係数を 256 フレーム (約 1.5 [s]^{*17}) ごとのセグメントに分割し, 学習を行う.

^{*16} $S(\mathbf{x})[f, t]$ は, $S(\mathbf{x})$ が周波数 f , 時刻 t の式として表されることを示す. 注意として, $S(\mathbf{x}) \in \mathbb{C}^{F \times T}$ であり, $S(\mathbf{x})[f, t] \in \mathbb{C}$ である.

^{*17} 開始フレーム間隔に基づく長さは $256 \times H/f_s \approx 1.486$ [s], 窓長を含めたカバー区間は $((256-1) \times H + L)/f_s = (255 \times 256 + 1024)/44100$ [Hz] ≈ 1.503 [s].

■ セグメント化

STFT 出力 $S(\mathbf{x}) \in \mathbb{C}^{F \times T}$ に対するセグメント化を, 時間フレーム方向の切り出し操作として定義する. 1 セグメントに含まれるフレーム数を $T_{\text{seg}} \in \mathbb{N}$ と定義し, 隣接するセグメント間のフレームシフト量を $H_{\text{seg}} \in \mathbb{N}$ とする. このとき, セグメントの開始フレームインデックスの集合 $\mathcal{T}$ は,

$$\mathcal{T} = \{t_0, t_1, t_2, \dots\} = \{t_0, t_0 + H_{\text{seg}}, t_0 + 2H_{\text{seg}}, \dots\}$$

で与えられ, 本研究では $T_{\text{seg}} = 256$ フレーム, セグメント重なり^{*18}は 50 [%], $H_{\text{seg}} = 128$ フレームとした. 従って, 隣接セグメントは 128 フレーム (約 0.74 [s]) 重なる. t_i は i 番目のセグメントが開始する時間フレーム番号を意味する. i 番目のセグメントを $\mathbf{X}^{(i)}$ と表記すると, これは開始フレーム t_i から連続する T_{seg} フレームを切り出すことで構成され, セグメント内部の相対フレームインデックス $\tau = 0, \dots, T_{\text{seg}} - 1$ に対して, 実部・虚部を分けて書けば

$$\Re \mathbf{X}^{(i)}[:, \tau] = \Re S(\mathbf{x})[:, t_i + \tau] \quad (9)$$

$$\Im \mathbf{X}^{(i)}[:, \tau] = \Im S(\mathbf{x})[:, t_i + \tau] \quad (10)$$

で与えられる. ここで, $:$ は全行を意味する.

末尾が不足する場合にはゼロパディング^{*19}を行い, 各セグメントを実部・虚部で並べた表現として $\mathbf{X}^{(i)} \in \mathbb{R}^{2 \times F \times T_{\text{seg}}}$ としてバッチ化する.

3.2.4 周波数領域の画像 f

STFT 出力 $S(\mathbf{x}) \in \mathbb{C}^{F \times T}$ に対し, 周波数領域の補正を表す画像を f で表す. f は入力スペクトルの周辺文脈を考慮する非線形画像として学習される. 本研究では, f をニューラルネットワークで実装し, 補正後のスペクトルを $f(S(\mathbf{x})) \in \mathbb{C}^{F \times T}$ として直接出力する.

3.2.5 逆短時間フーリエ変換 $S^\dagger$ (iSTFT)

STFT の逆変換は, 各フレームに対する逆 DFT と重ね合わせ (Overlap-Add) により与えられ, これを逆短時間フーリエ変換 (Inverse Short-Time Fourier Transform; iSTFT) と呼ぶ. この再構成画像を $S^\dagger$ と表す. 窓関数 w とホップ長 H が COLA (constant overlap-add) 条件を満たすとき, 適切な正規化の下で

$$S^\dagger(S(\mathbf{x})) = \mathbf{x} \quad (11)$$

が成立する [14].

Fig.1 に示すように, 本研究のネットワークは, $\hat{\mathbf{y}} = S^\dagger \circ f \circ S \mathbf{x}$ を $\mathbf{y}$ に近づけるように学習を行い, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ を最小化するように f を最適化する.

^{*18} STFT における窓関数のオーバーラップ率とは異なる.

^{*19} 信号の末尾にゼロを追加して長さを揃える処理.

4 データセットと前処理

4.1 収録条件

屋内で音源近傍に教師用マイクロホン、音源から離れた位置に入力用マイクロホンを配置し、同時に音声を収録した。汎用性の観点から、両マイクロホンともスマートフォン内蔵マイクロホンを用いており、サンプリング周波数は通常の 44.1 [kHz] で録音、ステレオ収録後にモノラルへ変換した。音源は会話やナレーションを含み、残響や環境ノイズ^{*20}が含まれている。データセットの総時間は約 20 [min] であり、うち約 16 [min] を学習用、約 4 [min] を評価用に用いた。

4.2 相互相関による時間揃え

4.2.1 相互相関を行う目的

収録を開始するタイミングにはどうしてもずれが生じてしまう。このずれが大きいと、学習時に教師信号と入力信号が時間領域で対応しなくなり、補正性能が著しく低下する。そのため、これらを学習前に補正するための前処理を行う必要がある。

4.2.2 相互相関 $R_{xy}(\tau)$ の定義

入力音声 $x \in \mathbb{R}^{N_x}$ と教師音声 $y \in \mathbb{R}^{N_y}$ に対し、音声 x を τ サンプルシフトさせたとき、どのくらい y と相関が高いかを示す相互相関 $R_{xy}(\tau)$ を以下のように定義する。

$$R_{xy}(\tau) = \sum_{i=0}^{N_y-1} \tilde{x}[\tau+i] \tilde{y}[i], \quad (12)$$

$$\tau = -(N_y - 1), \dots, N_x - 1$$

ここで $\tilde{x}[n] = x[n] - \bar{x}$, $\tilde{y}[n] = y[n] - \bar{y}$ は平均を引いた波形であり、配列外参照は 0 と見做す。実装では `numpy.correlate(tgt, ref, mode="full")` を用い、出力の k 番目の値は

$$R_{xy}[k] = \sum_{i=0}^{N_y-1} \tilde{x}[k - (N_y - 1) + i] \tilde{y}[i], \quad (13)$$

$$k = 0, \dots, N_x + N_y - 2$$

で与えられる ($\tau = k - (N_y - 1)$)。最も相関が高いシフト量 $\hat{\tau} = \underset{\tau}{\operatorname{argmax}} R_{xy}(\tau)$ を求め、これに基づき 2 系列の開始位置を揃え、共通区間のみを切り出す。

4.2.3 共通区間の切り出し

推定したラグ $\hat{\tau}$ を用いて、共通部分を得るために以下のように切り出す。

$$x', y' = \begin{cases} \tilde{x}[\hat{\tau}:], \tilde{y}[0:] & (\hat{\tau} \geq 0), \\ \tilde{x}[0:], \tilde{y}[-\hat{\tau}:] & (\hat{\tau} < 0) \end{cases} \quad (14)$$

^{*20} エアコン音や外部からの雑音など。

さらに、長さを小さい方に合わせて、

$$K := \min\{|x'|, |y'|\} \quad (15)$$

とし、

$$x \leftarrow x'[0:K], \quad y \leftarrow y'[0:K] \quad (16)$$

のように切り揃える^{*21 *22}。

この処理により、教師と入力音声が同一時間軸上で、かつ長さが整合したペアが得られる。なお $\hat{\tau}$ はサンプル数であり、44.1 [kHz] では 1 サンプルが約 22.7 [μ s] に相当する^{*23}。切り揃えた後の波形は、符号付き 32 [bit] 浮動小数点で保持し、最大振幅で正規化してから周波数変換を行う。

4.2.4 プログラム上での実装

前処理スクリプトでは、ライブラリ `librosa` で音声を読み込み、`numpy.correlate` で $R_{xy}(\tau)$ を求め、式 (12) に従って $\hat{\tau}$ を算出し、`soundfile.write` で整列済みのペアを保存する。相互相関の計算では事前に平均値を除算し、録音環境の定数分のオフセット^{*24}を除去している。`mode="full"` を用いることで、 $\tau = -(N_y - 1), \dots, N_x - 1$ の範囲を一度に評価する。以下 Listing 1, 2, 3 に主要な関数を示す (一部省略)。

Listing 1 相互相関による整列

```
def process_pair(input_path, teacher_path):
    """
    align input and teacher audio
    based on cross-correlation
    """
    # load audio files
    vec_x = load_audio(input_path) # x
    vec_y = load_audio(teacher_path) # y

    #  $\hat{\tau} \leftarrow \underset{\tau}{\operatorname{argmax}} R_{xy}(\tau)$ 
    lag = find_best_alignment(vec_x, vec_y)

    #  $(x, y) \leftarrow \text{crop}(x, y, \hat{\tau})$ 
    cropped_x, cropped_y = crop_common_region(
        vec_x, vec_y, lag
    )

    # save aligned and cropped audio
    sf.write(
        teacher_cropped_path, cropped_y, SR_RATE
    )
    sf.write(
        input_cropped_path, cropped_x, SR_RATE
    )
```

^{*21} この K は前章までの記号と同じ意味を成す。

^{*22} x, y はすでに定義済みのため、これを上書きすることを明示するために $\leftarrow$ を用いている。

^{*23} 1 [s]/44100 $\approx$ 22.7 [μ s]。

^{*24} 端末や録音環境による音量レベルの違いを指す。

Listing 2 整列用関数

```
def find_best_alignment(tgt: np.ndarray, ref:
    np.ndarray) -> int:
    """
    find lag tau which maximizes R_xy(tau)
    """
    # adjust mean to zero
    tgt = tgt - np.mean(tgt) #  $\tilde{x} \leftarrow x - \bar{x}$ 
    ref = ref - np.mean(ref) #  $\tilde{y} \leftarrow y - \bar{y}$ 

    #  $R_{xy}(\tau) \leftarrow \sum_i x[\tau + i] y[i]$ 
    corr = np.correlate(tgt, ref, mode="full")

    #  $\hat{\tau} \leftarrow \operatorname{argmax}_{\tau} R_{xy}(\tau)$ 
    imax = np.argmax(corr)

    #  $\hat{\tau} \geq 0$ : tgt[lag:] = ref[0:]
    #  $\hat{\tau} < 0$ : tgt[0:] = ref[-lag:]
    lag = imax - (len(ref) - 1)

    return lag
```

Listing 3 共通領域の切り出し

```
def crop_common_region(
    tgt: np.ndarray, ref: np.ndarray, lag: int
) -> tuple[np.ndarray, np.ndarray]:
    """
    crop common region of tgt and ref
    based on lag (tau)
    """
    # get size of vector tgt and ref
    tgt_length, ref_length = len(tgt), len(ref)

    # tgt_start, ref_start >= 0
    if lag >= 0:
        tgt_start = lag
        ref_start = 0
    else:
        tgt_start = 0
        ref_start = -lag

    # K: length of common region
    K = min(
        tgt_length - tgt_start,
        ref_length - ref_start
    )

    # crop common region
    tgt_cropped = tgt[tgt_start : tgt_start + K]
    ref_cropped = ref[ref_start : ref_start + K]

    return tgt_cropped, ref_cropped
```

なお、プログラム上で τ は lag, $\mathbf{x}$, $\mathbf{y}$ はそれぞれ tgt, ref のように表現している。実装では教師・入力ディレクトリ内の .wav ファイル名が一致することを事前に確認した。複数ペアの整列はペア単位で並列化し、Parallel^{*25}を用いて並列処理した。

^{*25} python ライブラリ joblib の Parallel [23] を指す。

5 モデル構成

5.1 モデルの概要と入出力形式

入力スペクトルと Hilbert 特徴の実部・虚部を 4 チャンネル画像として扱い、2 次元 U-Net で周波数領域の補正写像 f を学習する。

実信号 $x[n]$ に対する Hilbert 変換を

$$\mathcal{H}(x)[n] = \sum_{k \in \mathbb{Z}, k \neq 0} \frac{x[n-k]}{\pi k} \quad (17)$$

と定義し、本実装では $\mathcal{H}(x)$ を実数信号として扱い、その STFT を追加特徴として用いる。ここで $S(x) \in \mathbb{C}^{F \times T_{\text{seg}}}$ を $S(x) = \Re S(x) + j \Im S(x)$ と分解し、Hilbert 変換後の実信号 $\mathcal{H}(x)$ の STFT を $S(\mathcal{H}(x))$ と書けば、 $\mathbb{C}^{F \times T_{\text{seg}}} \simeq \mathbb{R}^{2 \times F \times T_{\text{seg}}}$ ^{*26} を 2 つ並べた

$$\mathbf{X} := \begin{bmatrix} \Re S(x) \\ \Im S(x) \\ \Re S(\mathcal{H}(x)) \\ \Im S(\mathcal{H}(x)) \end{bmatrix} \in \mathbb{R}^{4 \times F \times T_{\text{seg}}} \quad (18)$$

をネットワークの入力とする。

従って、実際の f は $S(x)$ の特徴量を増した $\mathbf{X}$ を入力とし^{*27}、補正後の実部・虚部に対応する 2 チャンネル画像 $f(\mathbf{X}) \in \mathbb{R}^{2 \times F \times T_{\text{seg}}}$ を出力する。補正後の複素スペクトルは $f(\mathbf{X}) \in \mathbb{R}^{2 \times F \times T_{\text{seg}}} \simeq \mathbb{C}^{F \times T_{\text{seg}}}$ として得る ($T_{\text{seg}} = 256$ フレーム)。

5.2 損失関数と最適化

5.2.1 損失関数の定義

ネットワークの出力 $f(\mathbf{X}) \in \mathbb{R}^{2 \times F \times T_{\text{seg}}}$ を複素数 $\mathbb{C}^{F \times T_{\text{seg}}}$ の元と見做す。このとき、推定波形は $\hat{\mathbf{y}} = S^\dagger(f(\mathbf{X}))$ と書け、逆に $f(\mathbf{X}) = S(\hat{\mathbf{y}})$ と書くこともできる。誤差関数は $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ を用いるが、具体的には、次式のように実部・虚部と振幅の両方を考慮する複素スペクトル損失 [7] と、Log-Mag 損失, SNR 損失, 無音強制項の線形和を用いる。

$$\begin{aligned} \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = & \|\Re S(\hat{\mathbf{y}}) - \Re S(\mathbf{y})\|_1 + \|\Im S(\hat{\mathbf{y}}) - \Im S(\mathbf{y})\|_1 \\ & + \lambda_{\log} \|\log(1 + |S(\hat{\mathbf{y}})|) - \log(1 + |S(\mathbf{y})|)\|_1 \\ & + \lambda_{\text{mag}} \| |S(\hat{\mathbf{y}})| - |S(\mathbf{y})| \|_1 \\ & + \lambda_{\text{snr}} \left(-10 \log_{10} \frac{\sum_{f,t} |S(\mathbf{y})[f,t]|^2}{\sum_{f,t} (|S(\mathbf{y})[f,t]| - |S(\hat{\mathbf{y}})[f,t]|)^2} \right) \\ & + \lambda_{\text{sil}} \sum_{f,t} \exp \left(- \frac{|S(\mathbf{y})[f,t]|}{\alpha_{\text{sil}}} \right) |S(\hat{\mathbf{y}})[f,t] - S(\mathbf{y})[f,t]| \end{aligned} \quad (19)$$

ここで $\|\cdot\|_1$ は L1 ノルムを表し、 $|\cdot|$ は複素数の絶対値 (振幅) を、 λ_{mag} , $\lambda_{\log}$, λ_{snr} , λ_{sil} は各損失の重み

^{*26} 複素数 $a + jb \in \mathbb{C}$ は、実数部 a と虚数部 b を並べた直積 $\mathbb{R} \times \mathbb{R}$ の元 $(a, b) \in \mathbb{R}^2$ と同一視できる。

^{*27} $\mathbf{X}$ は $S(x)$ の情報を完全に含んでおり、前章までは簡単のため、 $f(\mathbf{X})$ を $f(S(x))$ と表記している。

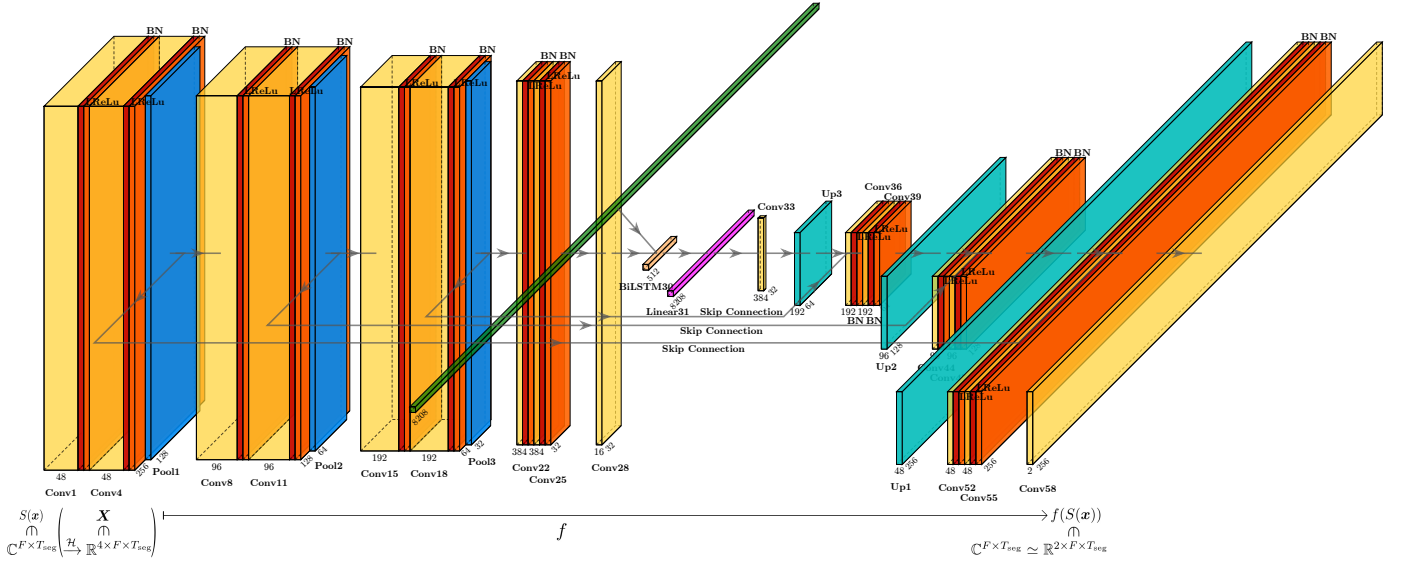


Fig. 4 本研究で用いた 2 次元 U-Net モデルの構成図。入力スペクトルと Hilbert 特徴の実部・虚部を 4 チャンネル画像として扱い、エンコーダ-デコーダ型の U-Net で周波数領域補正を行う。途中に LSTM ブロックを挟み、時間方向の長期依存性を考慮する。また、Skip Connection により、エンコーダの特徴マップをデコーダに直接渡すことで、詳細な情報を保持する。

を表す。また α_{sil} は、教師スペクトルの振幅が大きいほど無音強制項（差分項）の寄与が指数的に減衰するスケールを表す。これはそれぞれの損失の重要度を調整するためのハイパーパラメータであり、複素スペクトル損失に関しては、重さは 1 として固定され、最も重視される指標として扱い、他の 4 つの損失は補助的な役割を果たす^{*28}。

5.2.2 各損失の役割

5.2.2.1 複素スペクトル損失

複素スペクトル損失は、実部・虚部の両方を考慮することで、位相情報も含めたスペクトル全体の誤差を最小化する。具体的には $\|\Re S(\hat{y}) - \Re S(y)\|_1 + \|\Im S(\hat{y}) - \Im S(y)\|_1$ の項で、時間-周波数平面の複素成分の差を直接評価している。これにより、位相のずれによる音質劣化を抑制できる。実用上は、子音やアタック音^{*29}の立ち上がり、残響の尾部など、位相が影響しやすい時間構造の歪みを低減し、波形としての自然さを保つ効果が期待できる。

5.2.2.2 Log-Mag 損失

Log-Mag 損失は、振幅スペクトルの対数を用いることで、人間の聴覚特性に近い誤差評価を実現する。本研究では、振幅差と対数振幅差を合わせた指標として $\lambda_{\log} \|\log(1 + |S(\hat{y})|) - \log(1 + |S(y)|)\|_1 + \lambda_{\text{mag}} \| |S(\hat{y})| - |S(y)| \|_1$ で定義する。前者は対数によって大振幅を圧縮し、相対的な差を重視する項であり、後者は線型スケールでの振幅差を補正する項である。ここで $+1$ は $\log(0)$ を避けるための安定化

項であり、微小振幅の差を有限な値として扱えるようにする^{*30}。対数スケールは小さな振幅変化に敏感であり、微細な音質改善に寄与している。実用面では、無音区間や弱い成分のノイズフロア低減、高域のかすれ・ざらつきの抑制に効きやすく、聞き疲れの少ないスペクトル形状に近づける効果がある。

5.2.2.3 SNR 損失

SNR^{*31}損失は、信号対雑音比を最大化することを目的とし、全体的な音質向上に寄与する。ここでは SNR 損失を SNR の負値 $-10 \log_{10} \frac{\sum_{f,t} |S(y)[f,t]|^2}{\sum_{f,t} (|S(y)[f,t]| - |S(\hat{y})[f,t]|)^2}$ で定義し、スペクトル振幅の残差に対する信号エネルギー比を評価している。ここで $\sum_{f,t} \cdot$ は全ての周波数ビン f と時間フレーム t にわたる総和であるため、 $\sum_f \sum_t \cdot$ と同義である。係数の 10 はパワー比をデシベル表記に変換するためであり、 $\log_{10}$ を用いることで SNR の慣用的な尺度と一致させている。これにより、ノイズ成分の抑制と信号成分の強調が期待される。実用上では、信号全体の SNR 改善を直接意識した学習になるため、可視的なスペクトログラムの差だけでなく、録音としての明瞭度・聴き取りやすさの向上に繋がりやすい。

5.2.2.4 無音強制項

無音強制項は、教師信号の振幅が小さい時間-周波数領域において、推定スペクトルが教師スペクトル

^{*28} 実際、 $\lambda_{\text{mag}}, \lambda_{\log}, \lambda_{\text{snr}}, \lambda_{\text{sil}} \ll 1$ として調整を行った。

^{*29} 楽器音や声の発音開始部分における急激な音圧変化。

^{*30} 0 地点での発散を避けるということ。

^{*31} Signal-to-Noise Ratio. 信号成分がノイズ成分に対して強いことを示す指標であり、高ければ高いほど良い。

ルに一致するように誤差を強調し、無音性を強制することを目的とする。

具体的には $\lambda_{\text{sil}} \sum_{f,t} \exp\left(-\frac{|S(\mathbf{y})[f,t]|}{\alpha_{\text{sil}}}\right) |S(\hat{\mathbf{y}})[f,t] - S(\mathbf{y})[f,t]|$ を加えることで、 $|S(\mathbf{y})[f,t]| \ll \alpha_{\text{sil}}$ のとき指数重みが1に近づき、教師が無音に近い領域ほど複素スペクトル差分 $|S(\hat{\mathbf{y}})[f,t] - S(\mathbf{y})[f,t]|$ を強く罰する。一方で $|S(\mathbf{y})[f,t]| \gg \alpha_{\text{sil}}$ の領域では重みが0に近づくため、大きな成分の再現は他の損失項に委ねられる。

5.2.3 最適化

λ_{mag} , λ_{log} , λ_{snr} , λ_{sil} は各損失の重みであり、本研究では $\lambda_{\text{mag}} = 0.005$, $\lambda_{\text{log}} = 0.09$, $\lambda_{\text{snr}} = 0.03$ を固定し、無音強制項は $\lambda_{\text{sil}} = 0.55$, $\alpha_{\text{sil}} = 0.35$ を用いた。 λ_{sil} と α_{sil} は、検証データにおいて無音区間の残留成分が低減されるように探索して決定した。最適化には Adam(学習率 $\eta = 1.0 \times 10^{-4}$) を用い、ReduceLROnPlateau(patience=5, factor=0.5) で学習率を減衰させた。推論時は、時間方向に重なりを持つパッチごとにスペクトルを推定し、平均化して全時間の推定スペクトルを合成することで縫い目の少ないスペクトルを復元する。最後に逆変換 $S^\dagger$ を行い、波形 $\hat{\mathbf{y}}$ を得る。

5.2.4 前処理 – 学習アルゴリズムのフロー

以上の前処理から学習までの流れを Algorithm 1 にまとめる。まず、eq. (12) の相互相関 $R_{\mathbf{x}\mathbf{y}}(\tau)$ を最大化するずらし $\hat{\tau}$ を推定し、crop により入力 $\mathbf{x}$ と教師 $\mathbf{y}$ の時間位置を整列させる。次に $\mathcal{H}(\cdot)$ による Hilbert 変換を用いて $\mathbf{x}$ の特徴を拡張し、 $\mathbf{x}$, $\mathbf{y}$, $\mathbf{x}_{\mathcal{H}}$ それぞれに、STFT $S(\cdot)$ を計算する。その後、セグメント長 T_{seg} とホップ長 H_{seg} に基づき、時間方向に等間隔でセグメント位置集合 $\mathcal{T} = \{t_0 + iH_{\text{seg}}\}_{i=0}^{\lfloor (T-T_{\text{seg}})/H_{\text{seg}} \rfloor} = \{t_i\}$ を構成し、実部・虚部をチャネル方向に連結した入力 $\mathbf{X}^{(i)}$ と教師 $\mathbf{Y}^{(i)}$ を切り出す。最後に、得られたセグメント集合 $\mathcal{D}$ に対して損失 $\mathcal{L}$ (5.2 節) を最小化するように f を最適化し、推定スペクトルを逆変換して波形を復元する。

5.3 期待リスク最小化

最終的な推定は、データ分布 $p(\mathbf{x}, \mathbf{y})$ に対する期待リスク

$$\min_{\hat{\mathbf{y}}} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim p} [\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})] \quad (20)$$

をサンプル平均で近似して最小化する形に相当する。ここで $p(\mathbf{x}, \mathbf{y})$ は入力と教師のペアが生成される分布^{*32}を表し、学習データ $\{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N$ はそこから

^{*32} $p(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y}$ を「入力 $\mathbf{x}$ と教師 $\mathbf{y}$ がその近傍に現れる確率」と解釈する。確率密度または確率質量関数としての意味で用いる。

Algorithm 1: 前処理 – 学習アルゴリズム

Input: paired signals $\{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N$,
segment length T_{seg} , hop H_{seg} ,
epochs E , batch size B ,
learning rate η

Output: trained correction mapping f

$\mathcal{D} \leftarrow \emptyset$ // segment dataset

for each pair $(\mathbf{x}, \mathbf{y})$ **do**

$\hat{\tau} \leftarrow \arg \max_{\tau} R_{\mathbf{x}\mathbf{y}}(\tau)$ // eq. (12)

$(\mathbf{x}, \mathbf{y}) \leftarrow \text{crop}(\mathbf{x}, \mathbf{y}, \hat{\tau})$ // align by $\hat{\tau}$

$\mathbf{x}_{\mathcal{H}} \leftarrow \mathcal{H}(\mathbf{x})$ // Hilbert transform

$S_{\mathbf{x}} \leftarrow S(\mathbf{x})$, $S_{\mathbf{y}} \leftarrow S(\mathbf{y})$, $S_{\mathbf{x}_{\mathcal{H}}} \leftarrow S(\mathbf{x}_{\mathcal{H}})$

for $t_i \in \mathcal{T}$ **do**

$\mathbf{X}^{(i)} \leftarrow [\Re S_{\mathbf{x}}[:, t_i : t_i + T_{\text{seg}}],$

$\Im S_{\mathbf{x}}[:, t_i : t_i + T_{\text{seg}}],$

$\Re S_{\mathbf{x}_{\mathcal{H}}}[:, t_i : t_i + T_{\text{seg}}],$

$\Im S_{\mathbf{x}_{\mathcal{H}}}[:, t_i : t_i + T_{\text{seg}}]]$

$\mathbf{Y}^{(i)} \leftarrow [\Re S_{\mathbf{y}}[:, t_i : t_i + T_{\text{seg}}],$

$\Im S_{\mathbf{y}}[:, t_i : t_i + T_{\text{seg}}]]$

$\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{X}^{(i)}, \mathbf{Y}^{(i)})\}$

Initialize f , set up Adam(η) and scheduler

for $e = 1, \dots, E$ **do**

for mini-batch $(\mathbf{X}, \mathbf{Y}) \subset \mathcal{D}$ (size B) **do**

$\mathcal{L} \leftarrow \mathcal{L}(\mathbf{Y}, f(\mathbf{X}))$ // sec. 5.2

Update f by gradient descent on $\mathcal{L}$

Update scheduler

$\hat{\mathbf{y}} = S^\dagger(f(\mathbf{X}))$

独立に得られると仮定する。理想的には分布上の平均損失を最小化したいが、実際には有限個のデータしか得られないため、経験リスク^{*33}で近似する。実装では各ペアから STFT を計算し、 T_{seg} で区切ったセグメント集合 $\mathcal{D} = \{(\mathbf{X}^{(i)}, \mathbf{Y}^{(i)})\}$ を構成する。各 $\mathbf{X}^{(i)} \in \mathbb{R}^{4 \times F \times T_{\text{seg}}}$ は入力特徴、 $\mathbf{Y}^{(i)} \in \mathbb{R}^{2 \times F \times T_{\text{seg}}}$ は教師の複素スペクトルであり、 $f(\mathbf{X}^{(i)})$ は $S(\hat{\mathbf{y}}^{(i)})$ 、 $\mathbf{Y}^{(i)}$ は $S(\mathbf{y}^{(i)})$ に対応する。このとき期待値は経験リスク

$$\frac{1}{|\mathcal{D}|} \sum_{(\mathbf{X}, \mathbf{Y}) \in \mathcal{D}} \mathcal{L}(\mathbf{Y}, f(\mathbf{X})) \quad (21)$$

で近似できる。学習では $\mathcal{D}$ からミニバッチ^{*34}を取り出し、その平均損失に基づいて勾配を更新することで、経験リスクの最小化を確率的に近似している。Listing 4 にその実装を示す。

^{*33} 観測データに対する損失の平均であり、期待値を標本平均で近似したものを指す。

^{*34} 訓練データの部分集合を用いて損失と勾配を計算する手続きで、全データを用いる計算量を抑えるために用いる。

Table 1 動作環境

項目	Windows	Ubuntu (WSL2) on Windows
OS	Windows 10 Home (22H2) x86_64	Ubuntu 24.04.3 LTS (WSL2) x86_64
Kernel	WIN32_NT 10.0.19045.6466	6.6.87.2-microsoft-standard-WSL2
CPU	Intel Core i7-10750H (12) @ 2.59 [GHz]	"
GPU	NVIDIA GeForce RTX 3070 Laptop GPU 7.84 [GiB]	"
Memory	15.90 [GiB]	7.9 [GiB]
Shell	PowerShell 7.5.4	zsh 5.9

Table 2 実験設定の主要ハイパーパラメータ

項目	設定
サンプリング周波数	44.1 [kHz]
STFT 窓長 / ホップ長 (サンプル)	1024 / 256 (窓: Hann)
セグメント長 / 重なり (フレーム)	256 / 50 [%] (128 フレーム)
バッチサイズ	3
最適化手法	Adam, 初期学習率 1.0×10^{-4}
スケジューラ	ReduceLROnPlateau (patience=5, factor=0.5)
エポック数	256
損失重み	$\lambda_{\text{mag}} = 0.005$, $\lambda_{\text{log}} = 0.09$, $\lambda_{\text{snr}} = 0.03$, $\lambda_{\text{sil}} = 0.55$, $\alpha_{\text{sil}} = 0.35$

Listing 4 ミニバッチ更新 (学習ループ)

```

# for each mini-batch
for source_batch, target_batch in train_loader:
    # Move mini-batch to device
    source_batch = source_batch.to(device)
    target_batch = target_batch.to(device)

    # Reset gradients
    optimizer.zero_grad()

    #  $f(\mathbf{X}) \in \mathbb{R}^{2 \times F \times T_{\text{seg}}} \rightarrow \mathbb{C}^{F \times T_{\text{seg}}}$ 
    est = model(source_batch)

    # 2ch -> complex ( $\mathbb{R}^{2 \times F \times T_{\text{seg}}} \rightarrow \mathbb{C}^{F \times T_{\text{seg}}}$ )
    #  $f(\mathbf{X})$  as element of  $\mathbb{C}^{F \times T_{\text{seg}}}$ 
    estimated_cplx = u.channels_to_complex(est)
    #  $\mathbf{Y} = S(\mathbf{y})$  as element of  $\mathbb{C}^{F \times T_{\text{seg}}}$ 
    target_cplx = u.channels_to_complex(
        target_batch
    )

    #  $\mathcal{L}(\mathbf{Y}, f(\mathbf{X}))$ 
    loss = criterion(estimated_cplx, target_cplx)

    # Backprop + update
    loss.backward()
    optimizer.step()

```

この $\mathcal{L}(\mathbf{Y}, f(\mathbf{X}))$ は $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ を STFT 領域で評価したものに一致する^{*35}.

^{*35} $\mathbf{Y} = S(\mathbf{y})$, $f(\mathbf{X}) = S(\hat{\mathbf{y}})$ であるため.

5.4 ネットワーク詳細

モデルの構成は Fig.4 に示す通りであり、エンコーダ-デコーダ型の U-Net[5] を基盤とする. 入力テンソルの形状は (4, 513, 256) であり, これは実部・虚部と Hilbert 特徴の実部・虚部を 4 チャンネルとして積み上げたものである.

各畳み込みブロックは, 畳み込み層 (Convolution; Conv), バッチ正規化層 (Batch Normalization; BN), 漏れ ReLU 活性化関数 (Leaky ReLU; LReLU) から構成され, エンコーダは「Conv(3×3) → BN → LReLU」を 2 回適用する. チャンネル数は 4 → 48 → 96 → 192 と増加し, 各段の後に (1×2) の MaxPool を挿入して時間方向のみを 1/2 に縮退させる ($T_{\text{seg}} = 256 \rightarrow 128 \rightarrow 64 \rightarrow 32$) ため, 周波数分解能を維持したまま時間方向の文脈を集約する.

ボトルネック^{*36}ではさらに 2 回の Conv(3×3) で 384 チャンネルに拡張し, 1×1 畳み込みで 384→16 へ圧縮した後, 特徴次元 16×513=8208 の系列に対して 2 層 BiLSTM(hidden=256, 双方向) と線型層を適用して時間方向の長期依存関係を捉える. その後, 1×1 畳み込みで 384 チャンネルへ戻す.

デコーダは転置畳み込み (1×2) で 384 → 192 → 96 → 48 と復元し, 対応するエンコーダ出力を Skip Connection で結合してから各段で Conv(3×3) → BN → LReLU を 2 回適用する. 最終段では 1×1 畳み込みで 2 チャンネルを生成し, 複素スペクトルの

^{*36} ネットワークの最も狭い部分の層を指す.

実部・虚部として出力する．全体のパラメータ数は 27,280,594 である．

6 実験設定

6.1 動作環境

モデルの実装は PyTorch を用い、学習・推論は Windows 10 上の Ubuntu (WSL2) 環境で CUDA 対応 GPU (NVIDIA GeForce RTX 3070 Laptop GPU) を用いて行った (Table1). WSL 上では Memory がホスト OS と共有されるため、Ubuntu 側で利用可能なメモリ量がホスト OS のメモリのおおよそ半分程度に制限される点に注意されたい．

6.2 データセットと学習設定

データには、相互相関による時間揃えと共通区間の切り出しを施した 50 ペアの実録音波形を用いた．各波形は最大振幅で正規化し、train/validation に分割して学習した (検証比率 0.2, 本データでは訓練 40 ペア, 検証 10 ペア)．主要なハイパーパラメータを Table2 に示す．学習では、入力波形の STFT (実部・虚部) に Hilbert 特徴の STFT (実部・虚部) を加えた 4 チャネル特徴をモデルへ入力し、教師スペクトル (実部・虚部) との誤差を最小化した．損失関数の比較として、複素スペクトル損失のみ、無音強制項なし重み付き損失、無音強制項付き損失の 3 条件で学習を行い、以下の評価指標を記録した．

6.2.1 複素スペクトル損失低減量 ΔE_{cplx}

複素スペクトル損失 (実部・虚部の L1 誤差) に対応する指標として

$$E_{\text{cplx}}(\mathbf{z}) := \|\Re S(\mathbf{z}) - \Re S(\mathbf{y})\|_1 + \|\Im S(\mathbf{z}) - \Im S(\mathbf{y})\|_1 \quad (22)$$

を定義し、入力 $\mathbf{x}$ を基準とした低減量を

$$\Delta E_{\text{cplx}} := E_{\text{cplx}}(\mathbf{x}) - E_{\text{cplx}}(\hat{\mathbf{y}}) \quad (23)$$

で記録した．この指標は、予測音声 $\mathbf{z}$ を教師音声 $\mathbf{y}$ に近づける能力を純粋に評価するものである．

6.2.2 SNR 改善量 ΔE_{snr}

また、入力 $\mathbf{x}$ をベースとした SNR 改善量も評価した．ここで SNR 評価を行う対象の形状は実ベクトルのため、前章 (5.2.2.3) で定義したものとは少し異なる

$$E_{\text{snr}}(\mathbf{z}) := 10 \log_{10} \frac{\sum_n y[n]^2}{\sum_n (y[n] - z[n])^2} \quad (24)$$

で定義し、このときの、SNR 改善量 ΔE_{snr} を

$$\Delta E_{\text{snr}} := E_{\text{snr}}(\hat{\mathbf{y}}) - E_{\text{snr}}(\mathbf{x}) \quad (25)$$

で記録した．なお、音源分離の評価では SDR[15] や SI-SDR[16] が広く用いられるが、本研究では実録音

補正のベースライン比較として SNR 改善量 ΔE_{snr} を指標とした．評価では、検証データに対して全長の推論を行い、SNR とスペクトル形状との比較を行った．

6.2.3 無音領域の残留成分低減量 ΔE_{sil}

さらに、教師スペクトルの振幅が小さい領域 (下位 10%) に限定した残留成分パワーの低減量 ΔE_{sil} も記録した．

ここで、閾値 τ_{sil} を教師振幅 $|S(\mathbf{y})|$ の下位 10% 点 (分位点) とし、 $\mathcal{M}_{\text{sil}} := \{(f, t) \mid |S(\mathbf{y})[f, t]| \leq \tau_{\text{sil}}\}$ を無音マスクと定義する．このとき、無音領域における残留成分パワー E_{sil} を

$$E_{\text{sil}}(\mathbf{z}) := 10 \log_{10} \left(\frac{1}{|\mathcal{M}_{\text{sil}}|} \sum_{(f,t) \in \mathcal{M}_{\text{sil}}} |S(\mathbf{z})[f, t]|^2 + \varepsilon \right) \quad (26)$$

で定義し、このときの ΔE_{sil} を

$$\Delta E_{\text{sil}} := E_{\text{sil}}(\mathbf{x}) - E_{\text{sil}}(\hat{\mathbf{y}}) \quad (27)$$

で記録した．

ΔE_{snr} , ΔE_{sil} , ΔE_{cplx} はいずれも値が大きいほど良好であり、改善が見られた場合に正の値を取る．

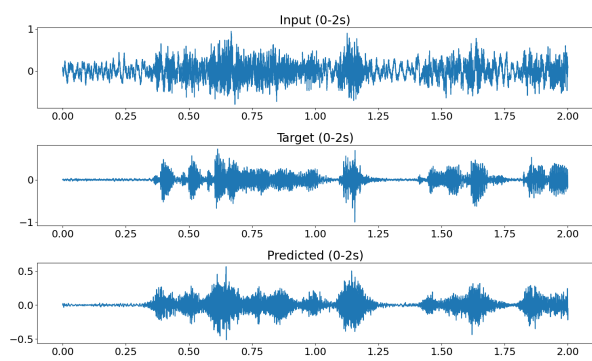
7 結果

7.1 複素スペクトル損失のみ

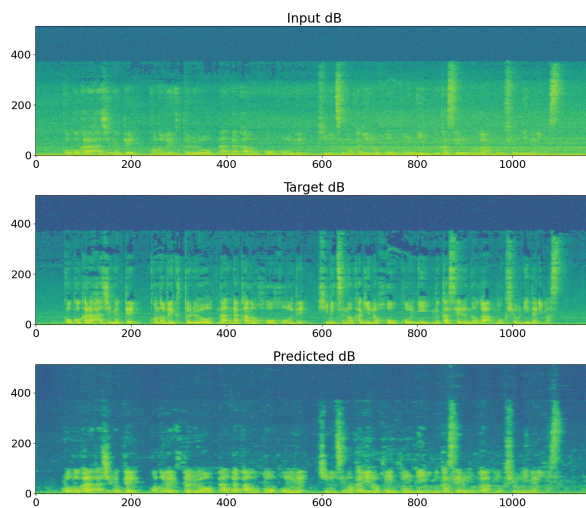
ここでは $\lambda_{\text{log}} = \lambda_{\text{mag}} = \lambda_{\text{snr}} = \lambda_{\text{sil}} = 0$ とした．学習曲線を Fig.8 に、検証データに対する時間波形と振幅スペクトログラムを Fig.5 に示す．当該例における複素スペクトル損失の低減量 ΔE_{cplx} は、0.666, 検証 10 ペア平均で 0.637 であった．また、SNR 改善量 ΔE_{snr} は、当該例で基準 -6.81 [dB] から -1.05 [dB] へと改善し、 5.77 [dB] の利得を得た．検証 10 ペアの平均では 5.17 [dB] の SNR 改善が得られた．さらに、教師スペクトルの振幅が小さい領域 (下位 10%) における残留成分パワーは、当該例で 19.14 [dB], 検証 10 ペア平均で 18.01 [dB] 低減した (Table3)．

7.2 無音強制項なし重み付き損失

ここでは $\lambda_{\text{log}} = 0.09$, $\lambda_{\text{mag}} = 0.005$, $\lambda_{\text{snr}} = 0.03$, $\lambda_{\text{sil}} = 0$ とした．学習曲線を Fig.9 に、検証データの一例に対する時間波形と振幅スペクトログラムを Fig.6 に示す．当該例における複素スペクトル損失の低減量 ΔE_{cplx} は、0.612, 検証 10 ペア平均で 0.586 であった．また、SNR 改善量 ΔE_{snr} は、基準 -6.81 [dB] から -1.07 [dB] へと改善し、 5.74 [dB] の利得を得た．検証 10 ペアの平均では 5.17 [dB] の SNR 改善が得られた．さらに、教師スペクトルの振幅が小さい領域 (下位 10%) における残留成分パワーは、当該例で 14.63 [dB], 検証 10 ペア平均で 12.88 [dB] 低減した (Table3)．

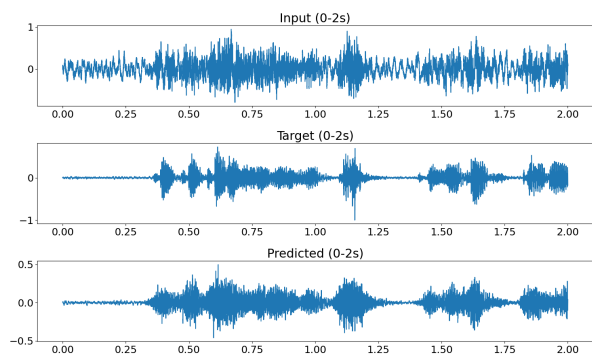


(a) 時間波形 (0 ~ 2 [s]).

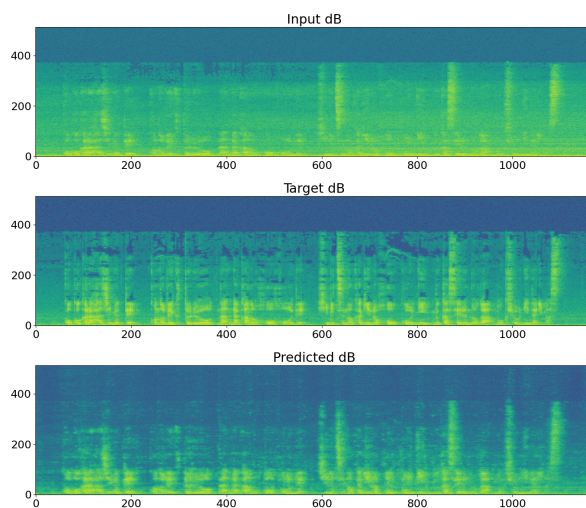


(b) 振幅スペクトログラム.

Fig. 5 推定音声の波形とスペクトル例 (複素スペクトル損失のみ).

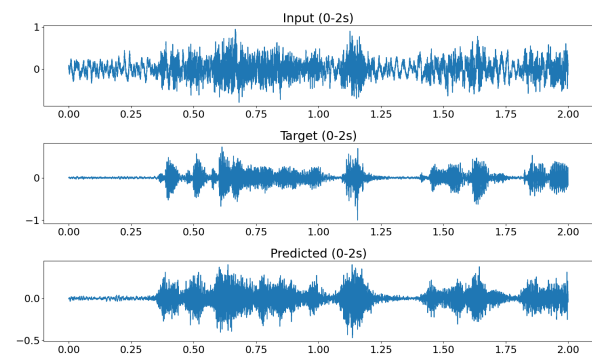


(a) 時間波形 (0 ~ 2 [s]).

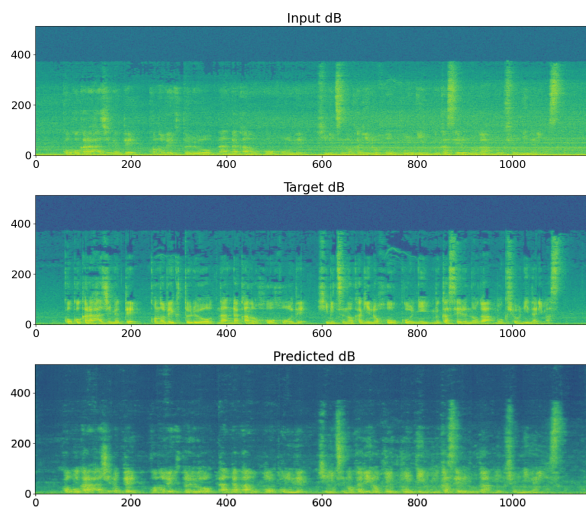


(b) 振幅スペクトログラム.

Fig. 6 推定音声の波形とスペクトル例 (重み付き損失).



(a) 時間波形 (0 ~ 2 [s]).



(b) 振幅スペクトログラム.

Fig. 7 推定音声の波形とスペクトル例 (無音強制項付き損失).

Table 3 損失函数による指標の比較 (単位:[dB], 複素スペクトル損失は単位なし).

$\lambda_{\log}$	λ_{mag}	λ_{snr}	λ_{sil}	α_{sil}	$\Delta E_{\text{cplx}}(\text{ex})$	$\Delta E_{\text{cplx}}(\text{avg})$	$\Delta E_{\text{snr}}(\text{ex})$	$\Delta E_{\text{snr}}(\text{avg})$	$\Delta E_{\text{sil}}(\text{ex})$	$\Delta E_{\text{sil}}(\text{avg})$
0	0	0	0	—	0.666	0.637	5.77	5.17	19.14	18.01
0.09	0.005	0.03	0	—	0.612	0.586	5.74	5.17	14.63	12.88
0.09	0.005	0.03	0.55	0.35	0.644	0.617	5.84	5.24	20.23	20.02

ex: 検証例, avg: 検証 10 ペア平均. ΔE_{cplx} は, 複素スペクトル損失 (実部・虚部の L1 誤差) の低減量 (23), ΔE_{snr} は, SNR 改善量 (25), ΔE_{sil} は, 教師振幅の下位 10% 領域における残留成分パワーの低減量 (27) を表す.

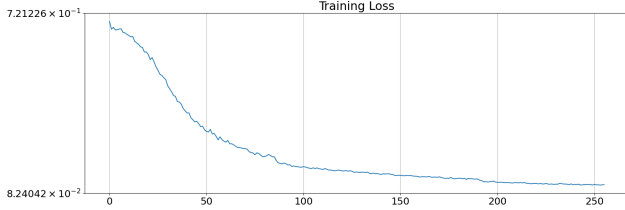


Fig. 8 学習損失の推移 (複素スペクトル損失のみ).

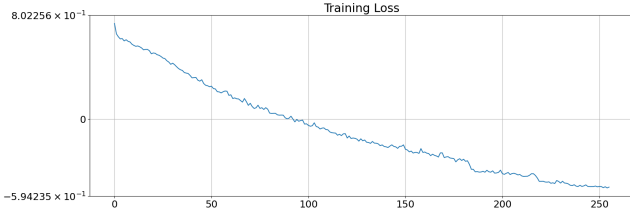


Fig. 9 学習損失の推移 (重み付き損失).

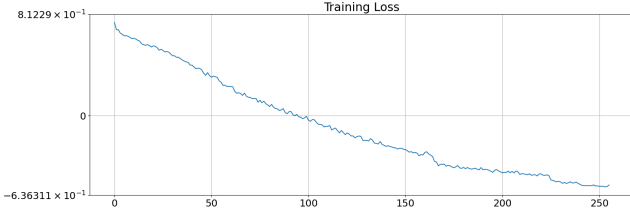


Fig. 10 学習損失の推移 (無音強制項付き損失).

7.3 無音強制項付き損失

ここでは $\lambda_{\log} = 0.09$, $\lambda_{\text{mag}} = 0.005$, $\lambda_{\text{snr}} = 0.03$, $\lambda_{\text{sil}} = 0.55$, $\alpha_{\text{sil}} = 0.35$ とした. 学習曲線を Fig.10 に, 検証データの一例に対する時間波形と振幅スペクトログラムを Fig.7 に示す. 当該例における複素スペクトル損失の低減量 ΔE_{cplx} は, 0.644, 検証 10 ペア平均で 0.617 であった. また, SNR 改善量 ΔE_{snr} は, 基準 -6.81 [dB] から -0.97 [dB] へと改善し, 5.84 [dB] の利得を得た. 検証 10 ペアの平均では 5.24 [dB] の SNR 改善が得られた. さらに, 教師スペクトルの振幅が小さい領域 (下位 10%) における残留成分パワーは, 当該例で 20.23 [dB], 検証 10 ペア平均で 20.02 [dB] 低減した (Table3).

7.3.1 損失函数の比較

Table3 に示す通り, SNR 改善は 5.2 [dB] 前後と概ね同程度であった (5.17 [dB]~ 5.24 [dB]). 一方で, 教師がほぼ無音の時間 - 周波数領域 (下位

10%) における残留成分パワーの低減量 ΔE_{sil} は, 12.9 – 20.0 [dB] 程度の差が見られ, 複素スペクトル損失のみ (18.01 [dB]) よりも重み付き (12.88 [dB]) が小さく, 無音強制項付き (20.02 [dB]) が最大となった. また, 複素スペクトル損失の低減量 ΔE_{cplx} は複素スペクトル損失のみが最大 (0.637) であり, 無音強制項付き (0.617), 重み付き (0.586) の順となった.

8 考察

8.1 全体性能と SNR 改善

50 ペア (訓練 40 ペア/検証 10 ペア) のデータセットに対して SNR が向上したことから, 相互相関による厳密な時間揃えと周波数領域補正が有効に機能したと考えられる. 損失函数の違いによる SNR 改善の差は小さいが, 複素スペクトル損失のみおよび重み付き (無音強制項なし) が 5.17 [dB], 無音強制項付きが 5.24 [dB] であった (Table3). 絶対値としては僅差であるものの, SNR においても無音強制項付きが最良であった点は興味深い. これは, 無音領域の残留成分抑制が全体の SNR 改善にも寄与し, 学習が幾分か促進された可能性が考えられる.

8.2 無音評価指標の導入と効果

本研究では, 教師がほぼ無音の時間 - 周波数領域 (下位 10%) に限定した残留成分パワー低減量 ΔE_{sil} を新たに記録し, 無音区間のノイズフロア抑制を評価できるようにした. この無音評価指標を導入したことで, 損失函数の選択・調整を「SNR のみ」から「SNR と無音強調の両立」へ拡張できた. その結果, 無音強制項付き損失は ΔE_{sil} が最大 (20.02 [dB]) となっただけでなく, SNR 改善も最大 (5.24 [dB]) となった (Table3). また, 複素スペクトル損失のみ (18.01 [dB]) と比べても無音強調が改善しており, 無音強制項が無音領域の残留成分抑制に有効であることが示唆される. 一方で, 複素スペクトル損失の低減量 ΔE_{cplx} は複素スペクトル損失のみが最大であり, 無音強制項付きでは僅かに低下した (Table3). これは, 無音付近の残留成分が波形誤差項に寄与し得るため, 無音強調の最適化が全体の複素スペクトル損失の最小化とトレードオフになる可能性があるからであると考えられる.

8.3 教師音声の残留ノイズによる影響

これまで、主に $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$ に基づく指標で学習・評価を行い、その中でも特に ΔE_{cplx} , ΔE_{snr} や ΔE_{sil} に注目して評価における議論を進めてきた。しかし、本研究で用いた教師音声にも収音系由来の環境ノイズが若干残っており、入力音声よりは高品質であるが、完全な無雑音信号ではない。教師を正解として学習する以上、この残留ノイズ成分もスペクトル形状の一部として再現され得るため、教師との比較に基づく指標が良くとも、人間が聴いた際の主観的な品質改善が必ずしも大きくはならない可能性がある。特に、入力よりもノイズが小さいものの、あるいは教師よりも強くノイズを抑えた場合に指標上の改善が小さく見える可能性がある。従って、Table3における教師基準の改善量が大きく増加しなかった点は、教師側に含まれる残留ノイズの影響も一因であると考えられる。

8.4 無音強制項の限界と今後の検証

一方で、無音に近い領域を強く抑えることは、弱い有用な成分(残響尾部や微小な倍音)まで消してしまうリスクもある。従って、無音強制項の有効性はSNRのような全体指標だけでは捉えにくく、主観評価や無音区間に特化した指標と併せて評価する必要がある。今回は無音領域の残留成分パワー低減量 ΔE_{sil} を導入し、手動で無音強制項の効果を確認したが、今後は無音強制項の重み λ_{sil} や形状パラメータ α_{sil} の影響を詳細に検証し、無音強調と弱成分保持のトレードオフを明確化することが望ましい。

9 結論と今後の課題

9.1 結論

本研究では、実録音音源の原音再現を目的として、STFTと2次元U-Netに基づく周波数領域補正型の音響フィルタを提案した。入力スペクトルを直接生成するのではなく、実録音スペクトルに加算する補正量を複素スペクトルとして推定し、iSTFTで時間波形を復元する構成を採用した。また、学習用対データを安定に構築するため、相互相関に基づく自動時間整列と共通区間の切り出しを行った。

50ペア(訓練40ペア/検証10ペア)の実録音データを用いて学習し、損失関数の比較を行った。検証データから10ペアを抽出して評価したところ、無音強制項付き損失がSNR改善量 ΔE_{snr} 及び無音領域の残留成分パワー低減量 ΔE_{sil} の両方で最良となり、平均でSNRを5.24[dB]改善し、教師振幅の低位10%領域における残留成分パワーを20.02[dB]低減した(Table3)。これより、無音強制項は全体的な復元性能を維持しつつ、無音区間のノイズフロア抑制に有効であることを確認した。

9.2 今後の課題

今後の課題として、評価の網羅性を高めるため、検証10ペア全体での指標集計と分散の集計、及びより多様な録音条件・音源への拡張を行う必要がある。また、無音強制項については、過大推定のみを罰する設計や重み関数の改良により、無音強調と弱成分保持のトレードオフを明確化することが望ましい。さらに、 > 10 [kHz] 帯でのガサつき低減を目的として、マルチスケールSTFT損失[22]やサブバンド別重み付け、モデル容量・正則化の調整を検討する。実用面では、モデルの軽量化と量子化、低遅延ストリーミング推論の検討を進めるとともに、MOS/ABX等の主観評価に加え、PESQ[17]やSTOI[18]等の客観指標を用いて品質を多面的に検証する必要がある。

謝辞

本研究を進めるにあたり、有明工業高等専門学校創造工学科人間・福祉工学系情報システムコースGauthier Lovic研究室のGauthier Lovic教授に多大なるご指導とご助言を賜った。ここに深く感謝申し上げます。

参考文献

- [1] Madam Aravind Kumar and Kamsali Manjunatha Chari, “Noise Reduction Using Modified Wiener Filter in Digital Hearing Aid for Speech Signal Enhancement,” *Journal of Intelligent Systems*, vol. 29, no. 1, pp. 1360–1378, 2020, doi:10.1515/jisys-2017-0509.
- [2] Georgios Ioannides and Vasilios Rallis, “Real-Time Speech Enhancement Using Spectral Subtraction with Minimum Statistics and Spectral Floor,” arXiv:2302.10313 [eess.AS], submitted Feb. 20, 2023.
- [3] J. Antoni and R. B. Randall, “Non-stationary signal processing for bearing health monitoring,” *Int. J. Manuf. Res.*, vol. 1, no. 1, pp. 18–40, 2006.
- [4] D. Wang and J. Chen, “Supervised speech separation based on deep learning: An overview,” *IEEE Trans. Audio Speech Lang. Process.*, vol. 26, no. 10, pp. 1702–1726, 2018.
- [5] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Proc. MICCAI*, 2015, pp. 234–241, doi:10.1007/978-3-319-24574-4_28.
- [6] N. J. Bryan and G. J. Scavone, “Performance of convolutional neural networks for musical

- source separation,” in *Proc. ICASSP*, 2018, pp. 71–75.
- [7] Y. Xiao *et al.*, “On complex neural network for spectral mapping,” in *Proc. Interspeech*, 2019, pp. 4604–4608.
 - [8] Y. Luo and N. Mesgarani, “Conv-TasNet: Surpassing ideal time–frequency masking for speech separation,” *IEEE Trans. Audio Speech Lang. Process.*, vol. 27, no. 8, pp. 1256–1266, 2019.
 - [9] A. Jansson *et al.*, “Singing Voice Separation with Deep U-Net Convolutional Networks,” in *Proc. ISMIR*, 2017, pp. 745–751.
 - [10] D. Stoller, S. Ewert, and S. Dixon, “Wave-U-Net: A Multi-Scale Neural Network for End-to-End Audio Source Separation,” in *Proc. ISMIR*, 2018, pp. 334–340.
 - [11] A. Défossez *et al.*, “Music source separation in the waveform domain,” *arXiv:1911.13254*, 2019.
 - [12] S. Pascual *et al.*, “SEGAN: Speech enhancement generative adversarial network,” in *Proc. Interspeech*, 2017, pp. 3642–3646.
 - [13] P. Gonzalez *et al.*, “Investigating the Design Space of Diffusion Models for Speech Enhancement,” *IEEE/ACM Trans. Audio Speech Lang. Process.*, vol. 32, pp. 4486–4500, 2024, doi:10.1109/TASLP.2024.3473319.
 - [14] J. B. Allen and L. R. Rabiner, “A unified approach to short-time Fourier analysis and synthesis,” *Proc. IEEE*, vol. 65, no. 11, pp. 1558–1564, 1977.
 - [15] E. Vincent, R. Gribonval, and C. Févotte, “Performance measurement in blind audio source separation,” *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 14, no. 4, pp. 1462–1469, 2006.
 - [16] J. Le Roux *et al.*, “SDR – half-baked or well done?” in *Proc. ICASSP*, 2019, pp. 626–630.
 - [17] ITU-T Recommendation P.862, “Perceptual evaluation of speech quality (PESQ): An objective method for end-to-end speech quality assessment of narrow-band telephone networks and speech codecs,” 2001.
 - [18] C. H. Taal, R. C. Hendriks, R. Heusdens, and J. Jensen, “A short-time objective intelligibility measure for time-frequency weighted noisy speech,” *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 19, no. 7, pp. 2125–2136, 2011.
 - [19] K. Choi *et al.*, “Phase-aware speech enhancement with deep complex U-Net,” in *Proc. ICLR Workshop*, 2019.
 - [20] Y. Hu *et al.*, “DCCRN: Deep Complex Convolution Recurrent Network for Phase-Aware Speech Enhancement,” in *Proc. Interspeech*, 2020, pp. 2472–2476.
 - [21] D. Yin, C. Luo, and Z. Xiong, “PHASEN: A Phase-and-Harmonics-Aware Speech Enhancement Network,” in *Proc. AAAI*, 2020.
 - [22] R. Yamamoto, E. Song, and J.-M. Kim, “Parallel WaveGAN: A fast waveform generation model based on generative adversarial networks with multi-resolution spectrogram,” in *Proc. ICASSP*, 2020, pp. 6199–6203.
 - [23] G. Varoquaux and O. Grisel, *joblib: running Python functions as pipeline jobs*, <https://joblib.readthedocs.io/>, accessed Jan. 2026.
 - [24] pytorch2tikz developers, *pytorch2tikz*, <https://pypi.org/project/pytorch2tikz/>, accessed Jan. 2026.